

// trabalho 2 · desenvolvimento mobile

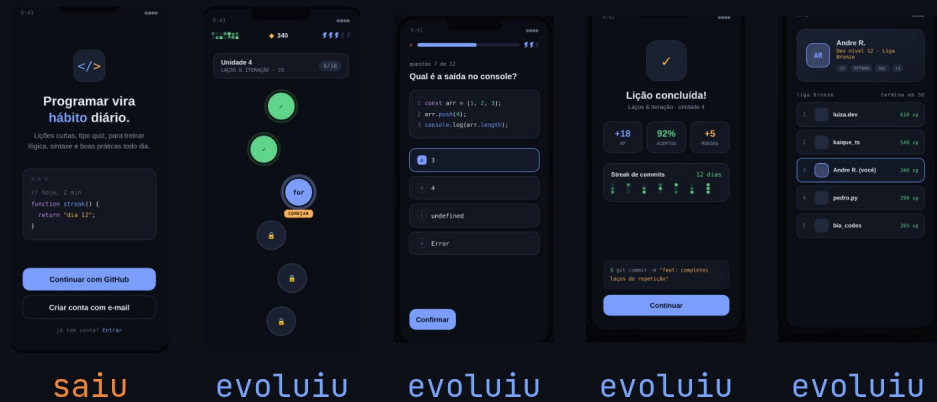
DuoCode

Navegação, listas e detalhes

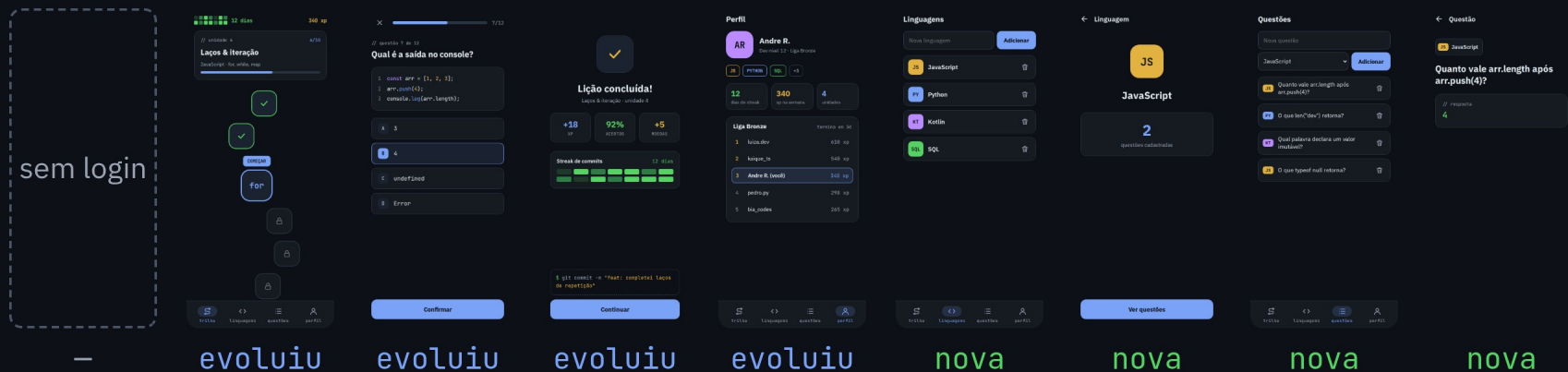
André Caires · Gustavo Cáceres · Brenno

O que mudou do Trabalho 1 para o 2

Trabalho 1
5 telas · protótipo



Trabalho 2
8 telas · app



Por que mudou

saiu

Login e onboarding

Por quê: a atividade não permite login. O app abre direto na Trilha.

Ganho: o foco do trabalho fica nas listas e na navegação.

evoluiu

Trilha, Quiz, Resultado e Perfil

O que mudou: Trilha e Perfil viraram abas do BottomNavigation. A Trilha ganhou barra de progresso na unidade, o Quiz um botão de sair e o Perfil cards de estatística.

Por quê: são o núcleo do app, então ficaram e se adaptaram à navegação nova.

nova

Linguagens, Questões e os 2 detalhes

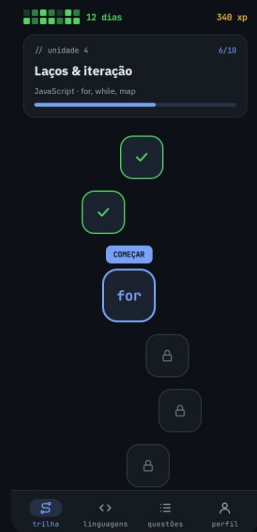
Por quê: o trabalho pede 2 listas com 2 detalhes.

Por que essas: vêm do próprio tema: toda questão pertence a uma linguagem, e o detalhe cruza as duas listas.

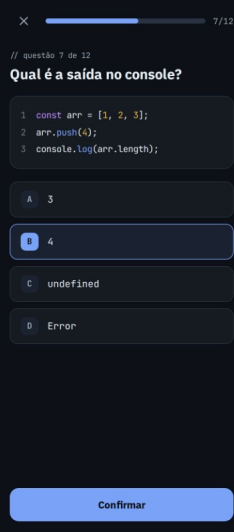
As 8 telas

fluxo de estudo

listas + detalhes (novas)



Trilha



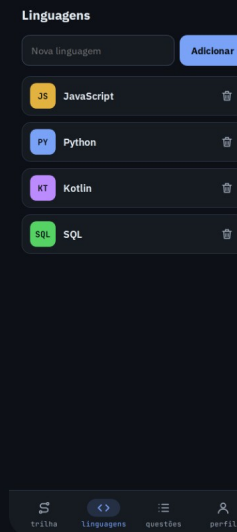
Quiz



Resultado



Perfil



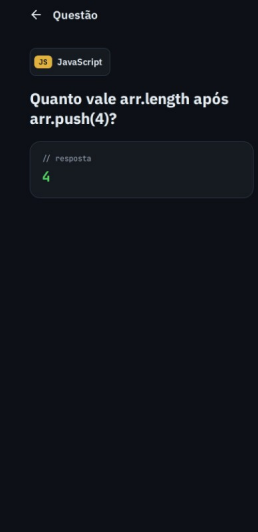
Linguagens



Det. ling.



Questões



Det. quest.

Fluxo de navegação

Fluxo de navegação

Caminho do usuário entre as 8 telas do DuCode



seta azul

Avançar: `navController.navigate(...)`

seta cinza

Voltar: `navController.popBackStack()`

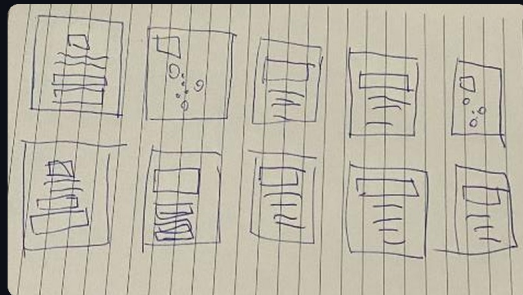
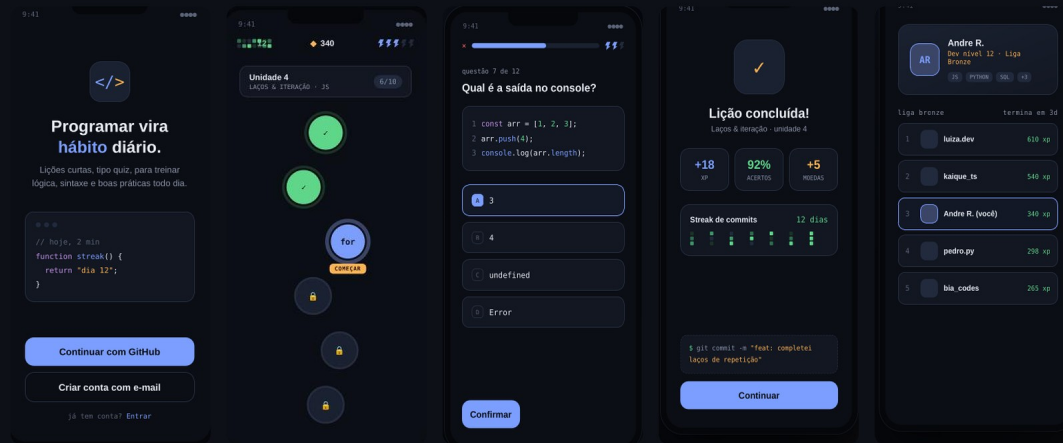
área tracejada

As 4 abas do BottomNavigation, troca livre entre elas

pergunta 1 de 5 • como estava no trabalho 1 e o que mudou

Do Trabalho 1 ao Trabalho 2

trabalho 1 • 5 telas no Canva, rascunho à mão e fluxo



saiu

A tela de login/onboarding

evoluíram

Trilha, Quiz, Resultado e Perfil

entraram

2 telas de lista e 2 telas de detalhe

novo na navegação

BottomNavigation, objeto Rotas e botão de voltar

duocode • trabalho 2

Por que essas telas novas

Linguagens

Faz: lista as linguagens, com adicionar e remover

Por quê: no DuoCode todo o conteúdo é organizado por linguagem

Questões

Faz: banco de perguntas, cada uma ligada a uma linguagem

Por quê: as questões são o núcleo do quiz

Detalhe da linguagem

Faz: mostra a linguagem e quantas questões ela tem

Por quê: deixa visível a relação entre as duas listas

Detalhe da questão

Faz: mostra enunciado, resposta e linguagem

Por quê: permite conferir uma questão antes de usar no quiz

// registro de decisões

Decisões do trio

O que decidimos, o que descartamos e por quê

Escopo do app

D1

Tirar a tela de login

descartamos

Manter o onboarding com cadastro por GitHub e e-mail

por quê

A atividade não permite login. O app abre direto na Trilha

D2

Simplificar as telas novas

descartamos

A primeira versão, com progresso, gráficos e responder a questão no detalhe

por quê

Era funcionalidade demais para o prazo. Ficamos com o que o enunciado pede

Dados

D3

Linguagem e Questao como data classes

descartamos

Outros pares, como Lição e Unidade

por quê

Vêm do próprio tema: toda questão pertence a uma linguagem. O linguagemId liga as duas

D4

Remover uma linguagem apaga as questões dela

descartamos

Deixar as questões sem linguagem

por quê

O detalhe da questão quebraria ao buscar uma linguagem que não existe mais

Navegação

D5

As listas ficam acima do NavHost

descartamos

Criar a lista dentro de cada tela

por quê

A tela de lista e a de detalhe precisam enxergar os mesmos dados

D6

Passar só o id pela rota

descartamos

Passar o objeto inteiro para o detalhe

por quê

A rota só leva texto simples. Com o id, o detalhe busca o item certo na lista

Como organizamos o código

objeto Rotas

Todas as rotas como const val String. Por quê: evita erro de digitação no nome da rota

NavHost

Um só NavHost em AppNavigation.kt controla as 8 telas

listas

As mutableStateListOf ficam no AppNavigation, acima do NavHost. Por quê: as listas e os dois detalhes enxergam os mesmos dados

```
AppNavigation.kt
navigation/AppNavigation.kt

58     val linguagens = remember {
59         mutableStateListOf()
60     }
61
62     val questoes = remember {
63         mutableStateListOf()
64     }
65
66     NavHost(
67         navController = navController,
68         startDestination = Rotas.LINGUAGENS,
69         modifier = Modifier.padding(innerPadding)
70     ) {
71         composable(Rotas.TRILHA) { TrilhaScreen(navController) }
72         composable(Rotas.PERGUNTA) { PerguntaScreen(navController) }
73         composable(
74             route = Rotas.RESULTADO,
75             arguments = listOf(
76                 navArgument("acertos") { type = NavType.IntType },
77                 navArgument("total") { type = NavType.IntType }
78             )
79         ) { backStackEntry ->
80             val acertos = backStackEntry.arguments?.getInt("acertos") ?: 0
81             val total = backStackEntry.arguments?.getInt("total") ?: 0
82             ResultadoScreen(navController, acertos, total)
83         }
84     }
85 }
```

Data classes e rotas

```
model/Models.kt

1 package com.example.duocode.model
2
3 data class Linguagem(val id: Int, val nome: String, val sigla: String, val descricao: String = "")
4
5 enum class Dificuldade(val rotulo: String) {
6     FACIL("fácil"),
7     MEDIO("médio"),
8     DIFICIL("difícil")
9 }
10
11 data class Questao(
12     val id: Int,
13     val enunciado: String,
14     val resposta: String,
15     val linguagemId: Int,
16     val dificuldade: Dificuldade = Dificuldade.FACIL,
17     val resolvida: Boolean = false
18 )
```

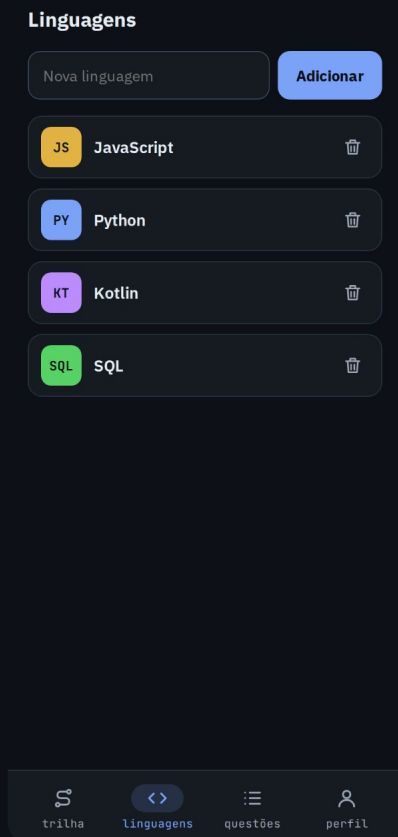
```
navigation/Rotas.kt

1 package com.example.duocode.navigation
2
3 object Rotas {
4     const val TRILHA = "trilha"
5     const val LINGUAGENS = "linguagens"
6     const val QUESTOES = "questoes"
7     const val PERFIL = "perfil"
8     const val DETALHE_LINGUAGEM = "detalhe_linguagem/{id}"
9     const val DETALHE_QUESTAO = "detalhe_questao/{id}"
10    const val PERGUNTA = "pergunta"
11    const val RESULTADO = "resultado/{acertos}/{total}"
12 }
```

ligação

linguagemId liga cada Questao à sua Linguagem

Lista de Linguagens



```
104         Button(
105             onClick = {
106                 val nome = novoNome.trim()
107                 if (nome.isNotEmpty()) {
108                     val novoId = (linguagens.maxOfOrNull { it.id } ?: 0) + 1
109                     val sigla = nome.take(2).uppercase()
110                     linguagens.add(Linguagem(id = novoId, nome = nome, sigla = sigla))
111                     novoNome = ""
112                 }
113             },
114             ...
115
116     LazyColumn(verticalArrangement = Arrangement.spacedBy(12.dp)) {
117         items(linguagens, key = { it.id }) { linguagem ->
118             Card(
119                 ...
120
121                 modifier = Modifier
122                     .fillMaxWidth()
123                     .clickable { navController.navigate("detalhe_linguagem/${linguagem.id}") }
124             ) {
125                 ...
126
127                 IconButton(onClick = {
128                     linguagens.remove(linguagem)
129                     questoes.removeAll { it.linguagemId == linguagem.id }
130                 }) {
```

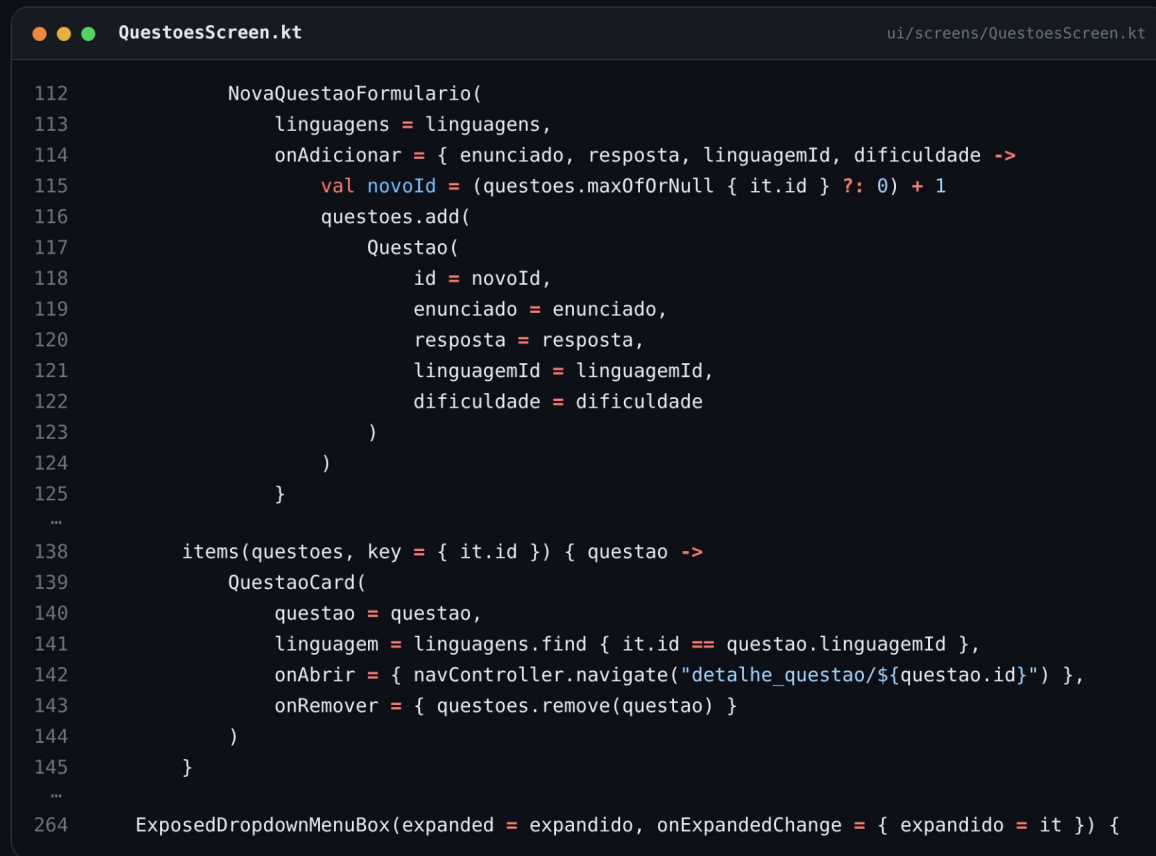
adicionar
linha 110

LazyColumn
linhas 125-126

abrir detalhe
linha 133

remover
linhas 165-166

Lista de Questões



adicionar
linhas 116-124

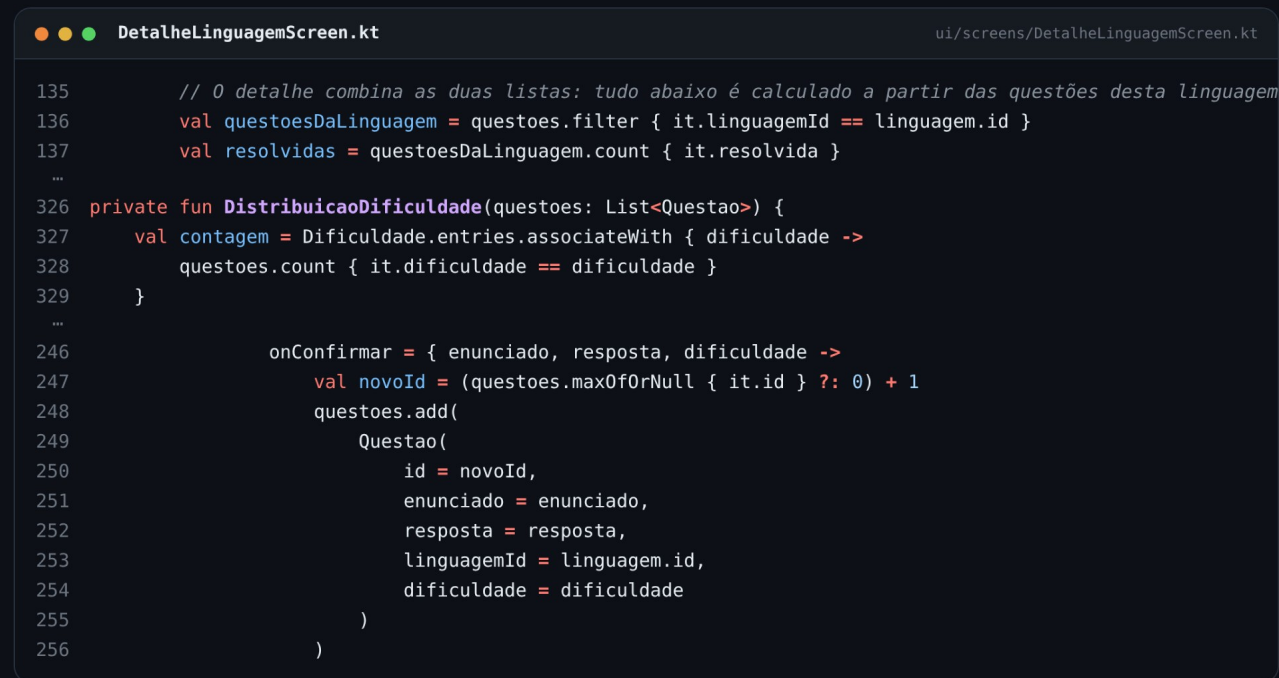
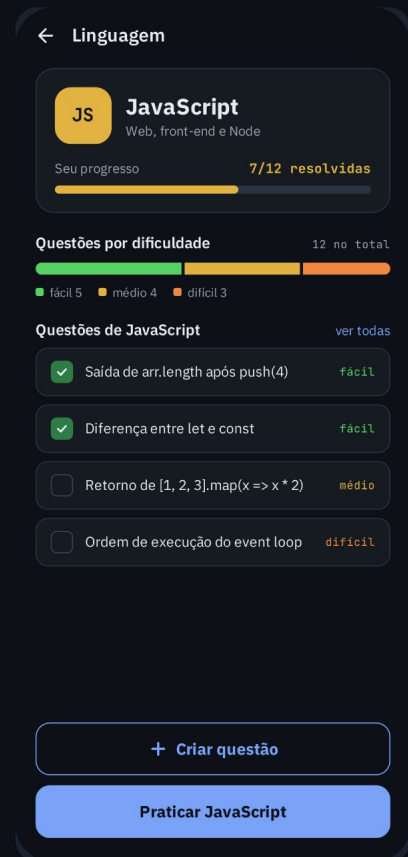
LazyColumn
linha 138

abrir detalhe
linha 142

remover
linha 143

dropdown
linha 264

Detalhe da linguagem: o algo a mais



progresso
linhas 136-137

dificuldade
linhas 326-329

criar questão
linhas 246-256

Por que essa: usa o linguagemId, a ligação entre as duas data classes. Toda questão pertence a uma linguagem.

Detalhe da questão

← Questão

JS JavaScript

Quanto vale arr.length após
arr.push(4)?

```
// resposta  
4
```

DetalheQuestaoScreen.kt

ui/screens/DetailQuestaoScreen.kt

```
76     val questao = questoes.find { it.id == id }  
...  
110     val linguagem = linguagens.find { it.id == questao.linguagemId }  
111     val alternarResolvida: () -> Unit = {  
112         val indice = questoes.indexOfFirst { it.id == questao.id }  
113         if (indice >= 0) {  
114             questoes[indice] = questao.copy(resolvida = !questao.resolvida)  
115         }  
116     }  
...  
124     if (linguagem != null) {  
125         val totalDaLinguagem = questoes.count { it.linguagemId == questao.linguagemId }  
...  
131         .fillMaxWidth()  
132         .clickable { navController.navigate("detalhe_linguagem/${linguagem.id}") }  
...  
228         Checkbox(  
229             checked = questao.resolvida,  
230             onCheckedChange = { alternarResolvida() },
```

busca pelo id
linha 76

linguagem dela
linhas 110 e 125

abre a linguagem
linha 132

resolvida
linhas 114 e 228

Fluxo de estudo: Trilha, Quiz e Resultado

PerguntaScreen.kt

ui/screens/PerguntaScreen.kt

```
79 fun PerguntaScreen(navController: NavController) {
80     var indiceAtual by remember { mutableIntStateOf(0) }
81     var selecionada by remember { mutableIntStateOf(-1) }
82     var acertos by remember { mutableIntStateOf(0) }
83
84     val total = perguntas.size
85     val pergunta = perguntas[indiceAtual]
86     val ultimaPergunta = indiceAtual == total - 1
87     ...
184     Button(
185         onClick = {
186             if (selecionada == pergunta.corretaIndex) acertos++
187             if (ultimaPergunta) {
188                 navController.navigate("resultado/$acertos/$total") {
189                     popUpTo(Rotas.TRILHA)
190                     launchSingleTop = true
191                 }
192             }
193         }
194     )
195 }
```

ResultadoScreen.kt

ui/screens/ResultadoScreen.kt

```
43 fun ResultadoScreen(navController: NavController, acertos: Int, total: Int) {
44     val percentual = if (total == 0) 0 else (acertos * 100) / total
45     val xpGanho = acertos * 6
46     val moedas = acertos
47 }
```

AppNavigation.kt

navigation/AppNavigation.kt

```
119     composable(
120         route = Rotas.RESULTADO,
121         arguments = listOf(
122             navArgument("acertos") { type = NavType.IntType },
123             navArgument("total") { type = NavType.IntType }
124         )
125     ) { backStackEntry ->
126         val acertos = backStackEntry.arguments?.getInt("acertos") ?: 0
127         val total = backStackEntry.arguments?.getInt("total") ?: 0
128         ResultadoScreen(navController, acertos, total)
129     }
```

rota com 2 argumentos

O Quiz conta os acertos e manda
resultado/{{acertos}}/{{total}}

resultado calculado

% de acertos e XP vêm dos números que chegaram pela rota

BottomNavigation e voltar

4 abas com navController.navigate(...)

```
AppNavigation.kt                                navigation/AppNavigation.kt

84         NavigationBar(containerColor = DuoCard) {
85             abas.forEach { aba ->
86                 val selecionada = rotaAtual?.hierarchy?.any { it.route == aba.rota } == true
87                 NavigationBarItem(
88                     selected = selecionada,
89                     onClick = {
90                         navController.navigate(aba.rota) {
91                             popUpTo(navController.graph.findStartDestination().id) {
92                                 saveState = true
93                             }
94                             launchSingleTop = true
95                             restoreState = true
96                         }
97                     },
```

Detalhes voltam com popBackStack()

```
DetalheLinguagemScreen.kt                       ui/screens/DetalheLinguagemScreen.kt

106         topBar = {
107             TopAppBar(
108                 title = { Text("Linguagem", fontWeight = FontWeight.Bold) },
109                 navigationIcon = {
110                     IconButton(onClick = { navController.popBackStack() }) {
111                         Icon(Icons.AutoMirrored.Filled.ArrowBack, contentDescription = "Voltar")
112                     }
113                 },
```

O código sendo construído

quando	quem	commit	o que entrou	linhas
02/10 · 14:07	André Caires	8423fc6	Estrutura (Rotas, AppNavigation, abas) + Perfil e Linguagens	+1.574
05/10 · 10:06	Gustavo Cáceres	231c725	Detalhe da linguagem: progresso, dificuldade e criar questão	+545 -9
05/10 · 10:21	Gustavo Cáceres	b3f4fac	Merge do pull request #1	—
05/10 · 13:24	Brenno Soares	3449c87	Tela de Questões: formulário, lista e remoção	+343 -1
05/10 · 13:26	Brenno Soares	41f083a	Merge do pull request #2	—
05/10 · 13:39	André Caires	205f158	Detalhe da questão: linguagem, resposta e resolvida	+282 -8
05/10 · 13:41	André Caires	e7edfd5	Merge do pull request #3	—
05/10 · 13:56	André Caires	104dc71	Trilha, Pergunta e Resultado evoluídas do Trabalho 1	+583 -19
05/10 · 13:58	André Caires	33a9736	Merge da branch telas-trabalho1	—

Desafios e como resolvemos

André

8423fc6 · 205f158

desafio

Montar a base que todos usariam e fazer a lista atualizar ao marcar uma questão como resolvida.

como resolveu

Abas com `popUpTo` e `saveState`. Para a lista perceber a mudança, troca o item inteiro por `questao.copy(resolvida = ...)`.

Gustavo

231c725

desafio

O detalhe precisava de dados que as data classes não tinham: dificuldade, resolvida e descrição.

como resolveu

Adicionou os campos com valor padrão e o enum `Dificuldade`, sem quebrar as telas que já existiam.

Brenno

3449c87

desafio

Escolher linguagem e dificuldade num formulário `Compose`. O protótipo nem tinha campo de resposta.

como resolveu

Criou um Seletor genérico com `ExposedDropDownMenuBox`, usado nos dois campos. O campo de resposta entrou depois.

Obrigado!

github.com/acairesm/Duo_Code_Parte_Dois

André Caires · Gustavo Cáceres · Brenno